

INPUT AND OUTPUT

Streaming Input

Understanding the two input modes for Claude Agent SDK and when to use each

Overview

The Claude Agent SDK supports two distinct input modes for interacting with agents:

- **Streaming Input Mode** (Default & Recommended) - A persistent, interactive session
- **Single Message Input** - One-shot queries that use session state and resuming

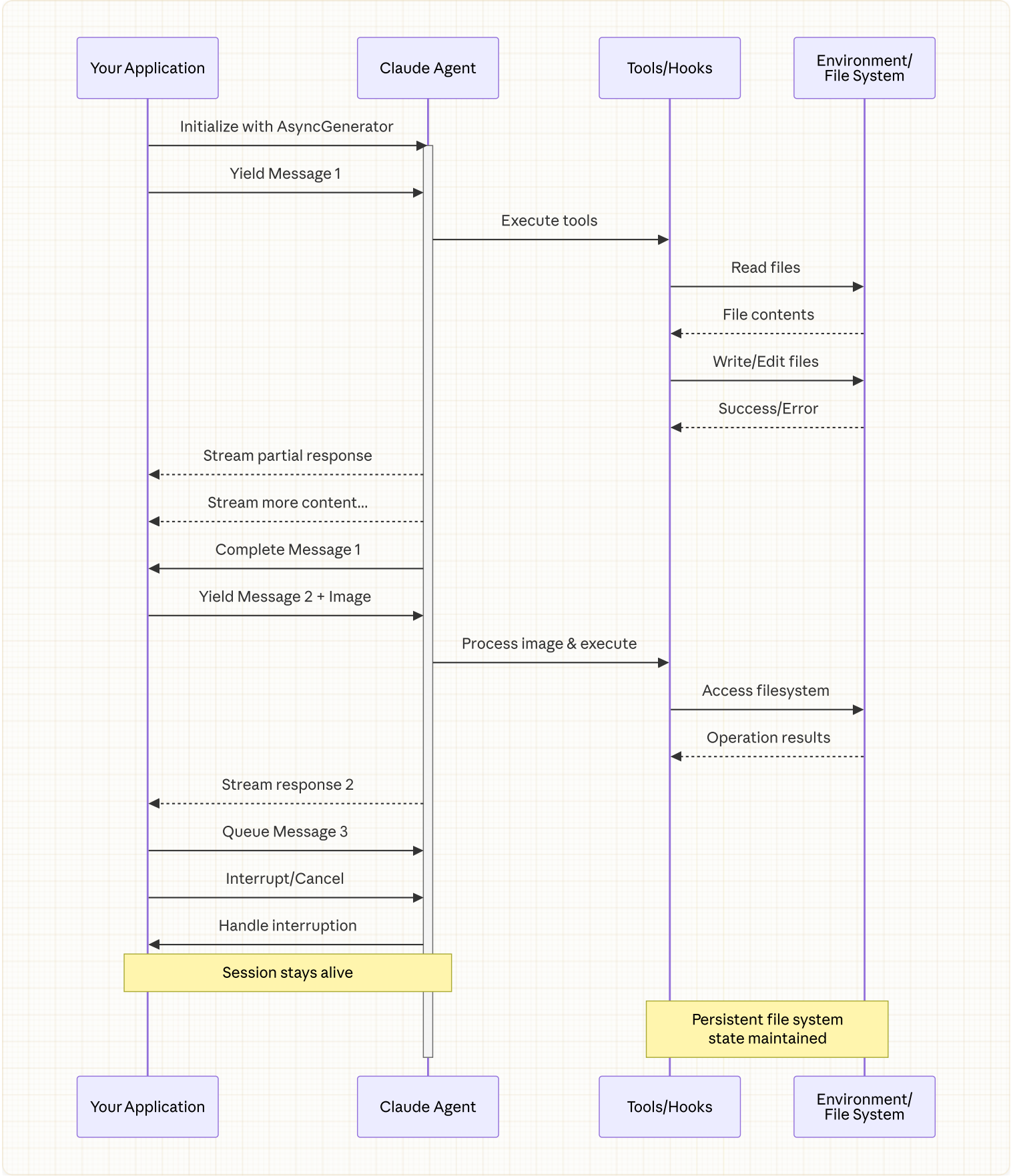
This guide explains the differences, benefits, and use cases for each mode to help you choose the right approach for your application.

Streaming Input Mode (Recommended)

Streaming input mode is the **preferred** way to use the Claude Agent SDK. It provides full access to the agent's capabilities and enables rich, interactive experiences.

It allows the agent to operate as a long lived process that takes in user input, handles interruptions, surfaces permission requests, and handles session management.

How It Works



Benefits

Image Uploads

Attach images directly to messages for visual analysis and understanding

Queued Messages

Send multiple messages that process sequentially, with ability to interrupt



Tool Integration

Full access to all tools and custom MCP servers during the session



Real-time Feedback

See responses as they're generated, not just final results



Context Persistence

Maintain conversation context across multiple turns naturally

Implementation Example

```
import { query, type SDKUserMessage } from "@anthropic-ai/claude-agent-sdk";
import { readFile } from "fs/promises";

async function* generateMessages(): AsyncGenerator<SDKUserMessage> {
  // First message
  yield {
    type: "user",
    message: {
      role: "user",
      content: "Analyze this codebase for security issues"
    },
    parent_tool_use_id: null
  };

  // Wait for conditions or user input
  await new Promise((resolve) => setTimeout(resolve, 2000));

  // Follow-up with image
  yield {
    type: "user",
    message: {
      role: "user",
      content: [
        {
          type: "text",
          text: "Review this architecture diagram"
        },
        {
          type: "image",
          source: {
            type: "base64",
            media_type: "image/png",
            data: await readFile("diagram.png", "base64")
          }
        }
      ]
    },
    parent_tool_use_id: null
  };
}

// Process streaming responses
```

```

for await (const message of query({
  prompt: generateMessages(),
  options: {
    maxTurns: 10,
    allowedTools: ["Read", "Grep"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}

```

❗ In the TypeScript SDK, if your message generator throws, for example when a file it reads is missing, the stream ends with an error that reads `Claude Code process aborted by user` instead of the original error, so check the code inside your generator first when you see that message. The error may also be preceded by a long minified line of bundled SDK source, so read to the end of the output for the error text.

In the Python SDK, a generator exception is logged at debug level and the session stalls without raising, so if a streaming session hangs with no output, enable debug logging and check your generator.

Single Message Input

Single message input is simpler but more limited.

When to Use Single Message Input

Use single message input when:

- You need a one-shot response
- You do not need image attachments or mid-session control methods
- You need to operate in a stateless environment, such as a lambda function

Limitations

⚠ Single message input mode does **not** support:

- Direct image attachments in messages
- Dynamic message queueing
- Real-time interruption
- Natural multi-turn conversations

If a query ends with an error result, such as `error_max_turns`, a single message `query()` call raises an error that includes the failure text after yielding the final result message, so wrap the loop in a try block if your code needs to continue. See [Handle the result](#) for the result subtypes.

Implementation Example

```
import { query } from "@anthropic-ai/claude-agent-sdk";

// Simple one-shot query
for await (const message of query({
  prompt: "Explain the authentication flow",
  options: {
    maxTurns: 1,
    allowedTools: ["Read", "Grep"]
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}

// Continue conversation with session management
for await (const message of query({
  prompt: "Now explain the authorization process",
  options: {
    continue: true,
    maxTurns: 1
  }
})) {
  if (message.type === "result" && message.subtype === "success") {
    console.log(message.result);
  }
}
```